

SAT-диспетчеризация сервисных инженеров

Полная и маршрутная модели,
математические гарантии
и сравнение с OR-Tools

3 набора · 45 запусков · настоящие дорожные матрицы

Полная SAT-модель: 1, 5, 30 и 120 секунд.

Быстрый SAT: лимит 1 секунда, ограничение усилий поиска.

Сравнение: Append, Insertion, Routing GLS и CP-SAT.

Граница достоверности данных

Дорожные времена и расстояния получены по графу OpenStreetMap через OSRM/FOSSGIS. Координаты заявок, база, навыки и смены инженеров остаются синтетическими. Это воспроизводимый вычислительный эксперимент, а не испытание реальной диспетчерской службы.

Дата измерений: 17.09.2026.

Apple M2 · 16 ГБ · один рабочий поток решателя.

OR-Tools 9.15.6755 · Rust release · CaDiCaL.

Аннотация

Представлены две реализации лексикографической диспетчеризации инженеров: полная инкрементальная SAT-модель с целочисленным кодированием времени и ленивым уточнением уличных маршрутов, а также быстрый SAT по ограниченному пулу заранее проверенных маршрутов. Приведены ограничения, булевы схемы, безопасные нижние оценки и область доказательства каждого режима. Выполнены 45 независимых запусков на трёх синтетических наборах из 66, 83 и 56 заявок с одинаковыми для методов реальными дорожными матрицами. Сравниваются два жадных алгоритма, SAT fast, SAT exact, OR-Tools Routing с GLS и CP-SAT. На Востоке exact достигает доказанного минимума шести инженеров начиная с 5 с; на Юго-востоке находит восемь без доказательства минимума; в Югоцентре GLS при 5–120 с даёт более короткие маршруты при том же штате, чем exact. Полный лексикографический оптимум не доказан ни в одном из 45 запусков. Результаты характеризуют эти реализации и фиксированные экземпляры, а не универсальное преимущество SAT или реальную потребность в персонале.

Ключевые слова: маршрутизация с временными окнами, неоднородные исполнители, лексикографическая оптимизация, SAT, кодирование порядка, ленивое уточнение, точное покрытие, CP-SAT.

Содержание

1. Задача и проверяемые утверждения	3
2. Математическая модель и методы решения	3
2.1. Данные, единицы измерения и область применимости	3
2.2. Допустимый план и лексикографическая цель	4
2.3. Полная SAT-модель: назначения и целочисленное время	4
2.4. Поэтапная оптимизация, схемы и смысл доказательства	7
2.5. Быстрый SAT по предварительно проверенным маршрутам	8
2.6. Эвристические решения, проверка и вычислительные ограничения	8
2.7. Детали булевой арифметики	9
3. Внешние методы и эквивалентность целей	10
3.1. Точная скаляризация для OR-Tools	10
3.2. Routing с Guided Local Search	10
3.3. Прямая модель CP-SAT	10
3.4. Контроль сопоставимости	11
4. Вычислительный эксперимент	12
4.1. Входные наборы и происхождение матриц	12
4.2. Протокол и измерение времени	12
4.3. Полные результаты	13
4.4. Сравнение и интерпретация	14
4.5. Скорость, размер модели и доказательства	15
4.6. Почему больше времени не всегда даёт лучшую строку	15
5. Ограничения и воспроизводимость	16
6. Источники и программные материалы	16

1. Задача и проверяемые утверждения

Прототип распределяет сервисные заявки между инженерами, учитывая навыки, транспорт, временные окна, длительности работ и уличные переезды. Главный приоритет — не минимизация километров: сначала минимизируются срочные пропуски, затем все пропуски, число уникальных инженеров за день и только после этого расстояние. Поэтому шесть инженеров должны выигрывать у семи при равном покрытии, даже при более длинных маршрутах.

Отчёт отвечает на три вопроса: какие допустимые планы действительно охватывает полная SAT-модель; что теряется при ограничении пула маршрутов в быстром режиме; как конкретные реализации ведут себя на одинаковых замороженных дорожных данных при ограниченном времени. Измеренные результаты не заменяются прежними офлайн-замерами или отдельно выбранным удачным пользовательским планом.

2. Математическая модель и методы решения

2.1. Данные, единицы измерения и область применимости

Рассматривается однодневное распределение заявок между инженерами с выбором порядка посещений. Множество заявок обозначим J , множество инженеров — E , множество срочных заявок — $J_U \subseteq J$. Заявка j имеет длительность обслуживания $p_j > 0$, окно **начала** обслуживания $[a_j, b_j]$, требуемый навык и, необязательно, требование к виду транспорта. Инженер e имеет исходную точку o_e , смену $[s_e, f_e]$, набор навыков и фиксированный транспортный профиль. Совместимость означает наличие навыка и совпадение транспорта, если он задан заявкой. Время измеряется целыми минутами от начала суток; расстояние — целыми метрами. Прототип принимает не более 100 заявок и 15 инженеров; границы окон и смен лежат в пределах 0–1440 минут.

CSV задаёт заявки, типы работ, адреса, районы и окна, но **не содержит проверенных GPS-координат и параметров персонала**. При импорте координаты детерминированно синтезируются около центров районов с зависящим от адреса смещением до 0,006 градуса по каждой координате. Это не геокодирование. Создаются 12 синтетических инженеров с общей базой в среднем арифметическом точек и сменой 09:00–23:59. Наборы навыков и транспорта назначаются циклически; в каждом цикле из шести инженеров три используют автомобиль, по одному — пеший, велосипедный и общественный транспорт. Поля контрольного распределения не задают ограничений и не являются эталоном оптимума.

Длительность p_j равна сумме технических работ и оформления документов из нормативов XLSX либо явно выбранного источника нормативов. Нормативная «дорога» не прибавляется: переезды учитываются отдельно. «Дозаказ» имеет собственный норматив, но требует навыка подключения; «Глобальная проблема» трактуется как срочная аварийная заявка. Явная длительность в JSON не заменяется нормативом.

В рабочем приложении уличный режим фиксирует направленные матрицы Valhalla по графу OpenStreetMap для профилей `auto`, `pedestrian`, `bicycle`. Описываемый ниже SAT-код не зависит от названия маршрутизатора: ему нужны согласованные матрицы времени и расстояния. В эксперименте настоящей статьи использованы отдельные дорожные профили OSRM/FOSSGIS для автомобиля, велосипеда и пешехода; происхождение и ограничения данных описывает Раздел 4. Для инженера e обозначим через τ_{ij}^e время и через d_{ij}^e расстояние перехода, включая переходы из o_e . Они зависят от направления и профиля; симметрия и неравенство треугольника не предполагаются. Длина — длина выданного маршрутизатором перехода, не обязательно кратчайшая по метрам. Если провайдер вернул время v в секундах и длину h в километрах, преобразование имеет вид

$$\tau = \left\lceil \frac{v + \varepsilon}{60} \right\rceil, \quad d = \lceil 1000h \rceil, \quad (1)$$

где $\varepsilon = 1$ секунда при $h > 0$ и $\varepsilon = 0$ иначе. Дополнительная секунда учитывает возможное усечение ответа провайдера. Недостижимый переход запрещён; в реализации он имеет временной маркер 100000, заведомо больший горизонта суток.

В приложении снимок Valhalla хранит также геометрию переходов. Экспериментальные OSRM-таблицы хранят только реальные дорожные стоимости, без геометрии, в отдельном кэше, не подключённом к интерфейсу. Поиск и проверка плана используют один замороженный снимок без сетевых запросов и без подмены недоступного перехода прямой линией. Матрицы статические: нет онлайн-пробок, зависимости времени от момента отправления и неопределённости длительностей. Профиль общественного транспорта в уличном режиме **не поддерживается** без расписаний GTFS; соответствующих инженеров необходимо явно исключить. Реальная уличная геометрия не превращает синтетические точки CSV в реальные

адресные координаты. Подтверждение точек интерфейсом также не является независимой геодезической проверкой. Описание формата матриц доступно в [документации Valhalla](#). Офлайн-режим с расстоянием по сфере и условными скоростями — отдельная демонстрационная модель, не источник уличных результатов.

2.2. Допустимый план и лексикографическая цель

Каждая заявка либо обслуживается ровно одним инженером, либо остаётся неназначенной. Инженер выполняет не более одного открытого маршрута $R_e = (j_1, \dots, j_r)$; пустой маршрут допустим. Возврат на базу не требуется и не входит в расстояние. Обслуживание непрерывно и не может перекрываться с другим обслуживанием или переездом того же инженера. Допускается ожидание до открытия окна. Для заданного порядка раннее расписание вычисляется рекуррентно:

$$t_{j_1} = \max(a_{j_1}, s_e + \tau_{o_e j_1}^e), \quad (2)$$

$$t_{j_{k+1}} = \max(a_{j_{k+1}}, t_{j_k} + p_{j_k} + \tau_{j_k j_{k+1}}^e). \quad (3)$$

Маршрут допустим тогда и только тогда, когда все заявки совместимы, все использованные переходы достижимы и

$$a_j \leq t_j \leq b_j, \quad t_j + p_j \leq f_e. \quad (4)$$

Таким образом, b_j ограничивает **начало**, а не окончание работы; окончание ограничено сменой. Ранний запуск достаточен для проверки фиксированного порядка: при статических неотрицательных временах более поздний запуск не может исправить нарушение верхней границы.

Пусть u_j — индикатор неназначения, y_e — индикатор непустого оставшегося маршрута, h_e — фиксированный признак того, что инженер уже был задействован в этот день. Минимизируется вектор

$$F = (U, V, H, D) \text{ лексикографически,} \quad (5)$$

$$U = \sum_{j \in J_U} u_j, \quad V = \sum_{j \in J} u_j, \quad (6)$$

$$H = \sum_{e \in E} h_e + \sum_{e \in E} (1 - h_e) y_e, \quad (7)$$

$$D = \sum_{e \in E} D_e, \quad D_e = d_{o_e j_1}^e + \sum_{k=1}^{r-1} d_{j_k j_{k+1}}^e \quad (8)$$

для непустого маршрута, а для пустого $D_e = 0$. Один дополнительный пропуск срочной заявки не компенсируется никаким улучшением остальных компонент. После фиксации срочных пропусков важнее число всех пропусков, затем число уникальных задействованных за день инженеров и лишь затем метры. Произвольная взвешенная сумма вместо этой иерархии не используется.

При событийном перепланировании приложение сохраняет префиксы с отправлением строго раньше времени события, включая начатый переезд, ожидание и обслуживание. Для остаточной задачи исходная точка переносится к последней сохранённой заявке, начало доступности — не раньше завершения префикса и времени события, а h_e устанавливается в единицу. Расстояние сохранённых префиксов — постоянная добавка к D и его границам. Доказательство для остаточной задачи условно относительно этих обязательств; оно не утверждает оптимальности задним числом за весь день.

2.3. Полная SAT-модель: назначения и целочисленное время

2.3.1. Безопасное сокращение областей

Для каждого транспортного профиля вычисляется кратчайшее по сумме времён замыкание матрицы снимка методом Флойда — Уоршелла. Обозначим его $\hat{\tau}_{ij}^e$. Замыкание допускает промежуточные точки снимка без обслуживания, поэтому

$$\hat{\tau}_{ij}^e \leq \tau_{ij}^e \quad (9)$$

и не превосходит времени любого составного перехода через эти точки. Оно используется **только как нижняя оценка**, а не вместо исходных переходов исполняемого маршрута.

Булева переменная x_{ej} означает назначение заявки инженеру. Она создаётся лишь для совместимой пары, для которой непуст интервал

$$L_{ej} = \max(a_j, s_e + \hat{\tau}_{o_e j}^e), \quad B_{ej} = \min(b_j, f_e - p_j). \quad (10)$$

Если $p_j > f_e$ или $L_{ej} > B_{ej}$, полагается $x_{ej} = 0$. Множество оставшихся пар обозначим A . Это необходимая проверка, а не требование достижимости заявки отдельным прямым выездом: в неметрической матрице промежуточное посещение может сделать её достижимой вовремя.

Основные ограничения имеют вид

$$u_j + \sum_{e:(e,j) \in A} x_{ej} = 1, \quad x_{ej} \leq y_e, \quad y_e \leq \sum_{j:(e,j) \in A} x_{ej}. \quad (11)$$

Эти ограничения точно задают смысл y_e : уже использованный инженер учитывается через константу h_e , даже если его остаточный маршрут пуст. Для назначенной пары также вводятся импликации

$$x_{ej} = 1 \Rightarrow L_{ej} \leq t_j \leq B_{ej}. \quad (12)$$

2.3.2. Монотонные пороги и предшествование

Для заявки с непустой областью назначений задаются

$$\ell_j = \min_{e:(e,j) \in A} L_{ej}, \quad r_j = \max_{e:(e,j) \in A} B_{ej}. \quad (13)$$

Единая переменная времени t_j достаточна, поскольку исполнитель единственен. Вместо двоичной записи времени используются пороговые литералы $Q_j(k)$ со смыслом $t_j \geq k$ для целых $\ell_j < k \leq r_j$:

$$Q_j(k+1) \leq Q_j(k), \quad t_j = \ell_j + \sum_{k=\ell_j+1}^{r_j} Q_j(k). \quad (14)$$

Вне области $Q_j(k) = 1$ при $k \leq \ell_j$ и $Q_j(k) = 0$ при $k > r_j$. У заявки без исполнителей время вырождено и не влияет на план. Целая минута остаётся точной сеткой: плотные интервальные отсечения ниже не укрупняют её.

Вспомогательный литерал P_{ij}^δ кодирует **одностороннее** условие

$$P_{ij}^\delta = 1 \Rightarrow t_j \geq t_i + \delta. \quad (15)$$

В CNF это граничные импликации и пороговые клаузы

$$\neg P_{ij}^\delta \vee \neg Q_i(k) \vee Q_j(k + \delta), \quad (16)$$

для существенных порогов, вместе с ограничениями $t_j \geq \ell_i + \delta$ и $t_i \leq r_j - \delta$ при активном P . Заведомо невозможное предшествование заменяется ложью, заведомо выполненное — истиной. Эквивалентность P и неравенства не требуется: при допустимых временах литерал можно активировать, но он не обязан быть истинным без внешнего требования. Одинаковые тройки (i, j, δ) используют общий литерал.

Для каждой пары заявок одного инженера вводится дизъюнкция, включаемая только их совместным назначением:

$$\neg x_{ei} \vee \neg x_{ej} \vee P_{ij}^{p_i + \hat{\tau}_{ij}^e} \vee P_{ji}^{p_j + \hat{\tau}_{ji}^e}. \quad (17)$$

Если интервалы уже гарантируют одну из альтернатив, клауза не нужна; невозможная альтернатива удаляется. Положительные p_j обеспечивают строгое различие времён начала у заявок одного инженера. До оптимизации расстояния отдельные переменные непосредственного следования не создаются.

2.3.3. Уличная релаксация и ленивое уточнение

При метрических временах сортировка назначенных заявок по t_j даёт допустимый порядок. Для уличной матрицы парные ограничения с $\hat{\tau}$ — только релаксация: SAT-ответ может использовать слишком оптимистичные составные переходы. Поэтому каждое найденное решение сначала сортируется по времени, а затем **полностью проигрывается** по исходным τ с ранним расписанием. Только успешно проигранный и проверенный план становится допустимой верхней оценкой.

Пусть у инженера e не прошёл проверку порядок $R = (j_1, \dots, j_r)$ с множеством заявок S . Добавляется запрет именно этого сочетания множества и порядка:

$$\left(\bigvee_{j \in S} \neg x_{ej} \right) \vee \left(\bigvee_{j:(e,j) \in A, j \notin S} x_{ej} \right) \vee \left(\bigvee_{k=1}^{r-1} P_{j_{k+1}j_k}^1 \right). \quad (18)$$

Последний дизъюнкт разрешает обратное предшествование хотя бы одной соседней пары старого порядка; его смысл — $t_{j_k} \geq t_{j_{k+1}} + 1$. Для одноэлементного маршрута этой части нет. Клауза включает и положительные литералы всех ранее невыбранных заявок данного инженера. Поэтому она **не запрещает надмножества**: добавление нового промежуточного посещения может исправить неметрический переход и должно оставаться допустимым. Один лишь запрет пары или запрет всех надмножеств здесь был бы некорректен.

Корректность уточнения. Для фиксированного множества всякий отличный строгий порядок меняет направление хотя бы одной соседней пары исходного порядка. Для исходного порядка раннее расписание уже нарушило окно или смену, а задержки не помогут. Значит, клауза исключает только недопустимое множество–порядок и сохраняет всякий реально допустимый маршрут. После добавления клауз решается та же задача с той же пробной границей цели. Такой SAT-ответ сам по себе ещё не понижает верхнюю границу. Возможных множеств и порядков конечное число, но их число комбинаторно велико; конечность не обещает завершения в коротком бюджете.

2.3.4. Плотные интервалы: обязательное перекрытие и счётные отсеечения

Для интервала $I = [\alpha, \beta]$ и обслуживания длительности p с началом в $[L, B]$ определим

$$g(v; I, p) = \max(0, \min(v + p, \beta) - \max(v, \alpha)), \quad (19)$$

$$w(L, B, p; I) = \min(g(L; I, p), g(B; I, p)). \quad (20)$$

Это точный минимум объёма обслуживания внутри I : функция перекрытия от начала обслуживания не имеет внутреннего минимума ниже значений на концах. Переезды в эту оценку не входят, что делает её оптимистичной, но безопасной.

Для инженера e положим $w_{ej} = w(L_{ej}, B_{ej}, p_j; I)$ и

$$C_e(I) = \max(0, \min(f_e, \beta) - \max(s_e, \alpha)). \quad (21)$$

Из непересечения обслуживаний следует необходимое энергетическое неравенство

$$\sum_{j: (e,j) \in A} w_{ej} x_{ej} \leq C_e(I). \quad (22)$$

В текущей CNF добавляется не вся эта взвешенная сумма, а семейство её кардинальных следствий. Для каждого различного положительного веса ω :

$$\sum_{j: (e,j) \in A, w_{ej} \geq \omega} x_{ej} \leq \left\lfloor \frac{C_e(I)}{\omega} \right\rfloor. \quad (23)$$

Они кодируются усечёнными унарными счётчиками и усиливают распространение ограничений на число совместимых по времени посещений.

Эти необязательные отсеечения строятся для уличной модели. Кандидатами левых границ служат a_j , правых — $b_j + p_j$. Для каждой левой границы выбирается интервал с максимальной плотностью суммы обязательных перекрытий всех заявок, затем сохраняется не более восьми наиболее плотных интервалов. Генерация ограничена счётчиком примерно 100000 добавленных вспомогательных переменных; проверка выполняется между целыми отсеечениями, поэтому последний счётчик может превысить порог. Ограничение объёма отсечений влияет только на скорость, не на допустимое множество.

Та же идея даёт нижнюю оценку числа инженеров после доказательства оптимального числа пропусков $V = q$. Для каждого интервала из границ $0, 1440, a_j, b_j + p_j$ берётся сумма $|J| - q$ **наименьших** значений $w(a_j, b_j, p_j; I)$; назовём её $W(I)$. Это наименьшая обязательная работа даже при самом выгодном выборе пропусков. У уже использованных инженеров суммируются все $C_e(I)$; мощности остальных сортируются по убыванию. Минимальное число дополнительных инженеров, покрывающее $W(I)$, вместе с $\sum_e h_e$ образует интервальную нижнюю оценку H . Максимум по просмотренным интервалам безопасен. Навыки, транспорт, срочность и переезды здесь игнорируются, то есть оценка может быть слабой, но не завышается.

2.3.5. Непосредственные переходы для цели по расстоянию

После доказательства первых трёх компонент цели добавляется полный слой допустимых дуг непосредственного следования, а не только дуги эвристического плана. Для каждого инженера создаются источник o_e , фиктивный сток z_e и булевы дуги $z_{o_e j}^e, z_{ij}^e, z_{j z_e}^e$. Сток не является географическим возвращением: его входящие дуги имеют нулевую стоимость и не требуют времени переезда.

Степенные ограничения:

$$\sum_j z_{o_e j}^e = y_e, \quad \sum_j z_{j z_e}^e = y_e, \quad (24)$$

$$z_{o_e j}^e + \sum_{i \neq j} z_{ij}^e = x_{ej}, \quad (25)$$

$$z_{j z_e}^e + \sum_{i \neq j} z_{ji}^e = x_{ej}. \quad (26)$$

В суммах участвуют только созданные дуги. Исходящие из источника дуги требуют $t_j \geq s_e + \tau_{o_ej}^e$; межзаявочные дуги требуют

$$z_{ij}^e = 1 \Rightarrow t_j \geq t_i + p_i + \tau_{ij}^e. \quad (27)$$

Дуга не создаётся, если безопасные нижняя и верхняя границы уже делают её невозможной. Все остальные направления допускаются, включая отсутствующие в стартовом решении. Окна и завершение работы в смену остаются ограничениями назначения.

Отсутствие подциклов. В любом выбранном межзаявочном переходе время строго возрастает, поскольку $p_i > 0$. Суммирование неравенств вокруг цикла даёт противоречие. При единичных входящих и исходящих степенях и единственном начале и конце все выбранные заявки образуют один путь от источника к стоку. Поэтому отдельное экспоненциальное семейство отсечений подциклов не требуется. Расстояние теперь выражается точно:

$$D = \sum_{e \in E} \left(\sum_j d_{o_ej}^e z_{o_ej}^e + \sum_{i \neq j} d_{ij}^e z_{ij}^e \right). \quad (28)$$

Копии дуг с одинаковыми концами и стоимостью у разных инженеров могут быть объединены одним литералом дизъюнкции в целевой сумме: одновременно активна не более одной такой копии, поскольку заявка имеет единственного исполнителя. Это сокращает арифметическую схему, но не набор переходов. Если верхняя граница D уже равна нулю, нижняя граница ноль сразу доказывает минимум и этот слой не нужен.

2.4. Поэтапная оптимизация, схемы и смысл доказательства

CNF решается инкрементально CaDiCaL. Ограничения «ровно один» состоят из дизъюнкции и «не более одного»; для коротких списков используются попарные клаузы, для длинных — линейное префиксное кодирование. Первые три цели полной модели используют унарные пороги счётчика. Если C_{rk} означает «среди первых r литералов истинны хотя бы k », то

$$C_{rk} \Leftrightarrow C_{r-1,k} \vee (v_r \wedge C_{r-1,k-1}), \quad (29)$$

с $C_{r0} = 1$ и невозможными порогами, равными нулю. Для границы K достаточно потребовать $\neg C_{r,K+1}$; ненужные старшие пороги не строятся. Нулевая цель задаётся непосредственно единичными отрицательными клаузами.

Взвешенная цель D кодируется деревом двоичных сумматоров с AND/XOR-элементами и компаратором с константой. Все переносы сохраняются: сумма не является арифметикой по модулю. Аналогичные взвешенные схемы используются в маршрутном режиме ниже и для единичных весов. Пробная граница передаётся временным литералом-предположением; неудачная попытка не закрепляет её навсегда.

На этапе k предыдущие компоненты уже доказаны и зафиксированы ограничениями сверху на найденные минимумы. Поддерживаются нижняя и верхняя границы $L_k \leq F_k^* \leq B_k$ в этой условной задаче. Начальная верхняя граница даётся только проверенным планом; нижняя обычно равна нулю, для персонала включает уже задействованных инженеров и интервальную оценку. Для U, V полная модель пробует середину диапазона, для H, D — $B_k - 1$.

- Проверенный SAT-кандидат с $F_k \leq K$ уменьшает верхнюю границу до его фактического значения. В уличной релаксации до этого выполняются проигрывание и, при необходимости, уточнение.
- Ответ UNSAT на $F_k \leq K$ даёт $L_k \leftarrow K + 1$. Он корректен и для релаксации: все реальные планы сохраняются её ограничениями и доказанно безопасными отсечениями.
- Прерывание, лимит времени или неопределённый ответ — **UNKNOWN**, а не UNSAT. Нижняя граница из такого ответа не повышается; возвращается лучший ранее проверенный план.

Следующий критерий оптимизируется только при $L_k = B_k$. Поэтому не смешиваются независимые покомпонентные минимумы, которые могли бы быть несовместимы. Полная лексикографическая оптимальность заявляется только после доказательства всех четырёх этапов. Недоказанные этапы не превращаются в доказанные из-за хорошего эвристического значения; исключение по существу одно — совпадение этого значения с уже обоснованной нижней границей.

Область утверждения. Полная модель охватывает все совместимые назначения и порядки для фиксированных целочисленных данных. Корректность сокращений, уточнений и слоя дуг связывает её ответы с допустимыми маршрутами. Однако текущая программа сообщает результат UNSAT решателя и численные границы, а не экспортируемый независимо проверенный сертификат доказательства. «Глобальный оптимум» относится к этой модели, не к неизвестным фактическим GPS-позициям, трафику или производительности работников.

2.5. Быстрый SAT по предварительно проверенным маршрутам

При $|J| \leq 16$ быстрый режим непосредственно вызывает полную модель; область доказательства тогда глобальная. Для более крупных задач формируется конечный пул $\mathcal{R}$ непустых маршрутов. Каждый маршрут r заранее проверяется по исходной матрице и имеет инженера $e(r)$, множество посещений S_r и расстояние D_r . Источники пула — базовый и вставочный жадные планы, их варианты с удалением одной заявки, допустимые одиночные выезды и ограниченное число детерминированных многостартовых планов. Каждый подмаршрут проверяется заново: в неметрической матрице удаление остановки может нарушить допустимость.

Для одного инженера и одного множества заявок сохраняется кратчайший **среди сгенерированных допустимых порядков**. Это корректное доминирование внутри пула: остальные критерии и конфликты покрытия совпадают, а межмаршрутных временных связей нет. Оно не означает нахождения кратчайшего порядка среди всех перестановок.

С булевой переменной выбора λ_r задача становится точным покрытием с разрешёнными пропусками:

$$u_j + \sum_{r \in \mathcal{R}: j \in S_r} \lambda_r = 1, \quad (30)$$

$$\sum_{r \in \mathcal{R}: e(r)=e} \lambda_r = y_e. \quad (31)$$

Следовательно, каждая заявка либо покрыта ровно одним маршрутом, либо пропущена, а инженер получает не более одного маршрута. Критерии U, V, H прежние; $D = \sum_{r \in \mathcal{R}} D_r \lambda_r$. Стартовый допустимый план представлен в пуле, а решение декодируется и снова проверяется общей процедурой.

Оптимизация также лексикографическая, но пробует последовательные улучшения $B_k - 1$. На каждый оптимизационный SAT-вызов установлен предел 500 конфликтов; достигнутый предел даёт UNKNOWN и останавливает дальнейшие этапы. Начальный вызов с предположениями стартового плана такого конфликтного ограничения не имеет, сохраняя временной дедлайн. Генерация многостартовых колонок ограничена 24 попытками и сроком не более $\min(0.25T, 0.05)$ секунды от начала запуска; обязательное начальное наполнение пула предшествует этой проверке срока.

Все нижние границы и доказательства быстрого режима при $|J| > 16$ относятся **только к пулу**. Его оптимум даёт допустимое решение исходной задачи, но его нижняя граница в общем случае не является глобальной нижней границей: отсутствующий маршрут может улучшить результат. Даже завершение всех этапов обозначается как оптимум пула, без утверждения глобальной оптимальности. Пул не является полным перечислением маршрутов и не расширяется ценообразованием, как в методе генерации столбцов.

2.6. Эвристические решения, проверка и вычислительные ограничения

Базовая жадная процедура обрабатывает заявки в исходном порядке и дописывает заявку первому инженеру, для которого такой маршрут допустим. Вставочная версия ставит срочные заявки раньше, затем ориентируется на конец и начало окна; перебирает позиции вставки, предпочитает не увеличивать число задействованных инженеров и уменьшать добавочное расстояние. Детерминированные многостарты меняют порядок внутри временных групп и предпочтения вставки.

Полная модель использует эти планы как допустимые начальные решения и верхние границы. Локальная полировка перебирает переносы, обмены, развороты фрагментов и попытки освободить целого инженера переносом его заявок. Она сохраняет множество обслуженных заявок и принимает лексикографические улучшения по персоналу и расстоянию. Все затронутые маршруты проигрываются полностью, а не оцениваются метрическими формулами изменения стоимости. Тёплый старт задаётся временными предположениями, не постоянными ограничениями назначения или времени. **Ни жадный план, ни локальная полировка не сужают пространство полной SAT-модели.** В быстром режиме эвристики, напротив, явно определяют неполное пространство маршрутных колонок; это ограничение уже отражено в области доказательства.

Общий валидатор независимо от SAT-декодирования проверяет совместимость, отсутствие повторных назначений, полноту разбиения на назначенные и пропущенные заявки, времена переезда, ожидания, окна начала, завершение в смену, расстояния и итоговые метрики. При наличии геометрии она сверяется с тем же снимком. Это независимое проигрывание маршрута, но не независимая проверка качества исходной карты или данных.

Заданный бюджет T — рабочий дедлайн, а не жёсткая гарантия времени ответа операционной системы. SAT использует callback остановки; часть подготовки, построения схем и финальной валидации выполняется между проверками времени. Базовый жадный план строится без собственного дедлайна. В полной

модели при более чем 16 заявках до $\min(0.1T, 0.1)$ секунды резервируется для заключительной полировки; короткие полировки возможны и после найденных SAT-решений. Поэтому необходимо отдельно фиксировать лимит и фактически измеренное время, а не считать их тождественными. При остановке во время построения CNF возвращается проверенное эвристическое решение без вымышленного доказательства. По размеру полная постановка содержит до $O(|E||J|)$ назначений, $O\left(\sum_{j(r_j-\ell_j)}\right)$ временных порогов и до $O(|E||J|^2)$ отношений/дуг, причём кодирование временных импликаций дополнительно зависит от ширины окон. Ленивая схема откладывает дорогостоящие дуги до цели расстояния, но не устраняет комбинаторную сложность. Быстрый режим меньше за счёт неполного пула, а не за счёт эквивалентного сокращения глобальной задачи. Модель не содержит перерывов, вместимости транспорта, запасов оборудования, нескольких смен или стохастических запасов времени, если они отдельно не отражены во входных данных.

2.7. Детали булевой арифметики

Для полноты ниже приведены используемые локальные CNF-схемы. Эквивалентность $q \Leftrightarrow (a \wedge b)$ задаётся тремя клаузами

$$(\neg q \vee a) \wedge (\neg q \vee b) \wedge (q \vee \neg a \vee \neg b). \quad (32)$$

Для $q \Leftrightarrow (a \oplus b)$ используются четыре клаузы

$$(\neg a \vee \neg b \vee \neg q) \wedge (a \vee b \vee \neg q) \wedge (a \vee \neg b \vee q) \wedge (\neg a \vee b \vee q). \quad (33)$$

Дизъюнкция реализуется через отрицание конъюнкции отрицаний. Константы, повторяющиеся и противоположные литералы упрощаются до построения схемы.

В полном сумматоре с входным переносом c_i :

$$v_i = a_i \oplus b_i, \quad z_i = v_i \oplus c_i, \quad c_{i+1} = (a_i \wedge b_i) \vee (v_i \wedge c_i). \quad (34)$$

Начальный перенос равен нулю, последний перенос сохраняется как старший разряд. Для слагаемого w в позиции i стоит литерал x , если соответствующий бит константы w равен единице, иначе ноль. Пары векторов складываются послойно, образуя дерево.

Для сравнения двоичных векторов $A \leq B$ разряды просматриваются от младшего к старшему:

$$\ell_{-1} = 1, \quad \ell_i = (\neg a_i \wedge b_i) \vee ((a_i \Leftrightarrow b_i) \wedge \ell_{i-1}). \quad (35)$$

Более значимый различающийся разряд переопределяет результат младших. Последний ℓ_i эквивалентен требуемому сравнению и используется как литерал-предположение пробной границы. Тем самым арифметика цели — точная целочисленная схема, а не внешняя оценка после SAT-поиска.

3. Внешние методы и эквивалентность целей

Сравниваются конкретные реализации, а не абстрактные классы алгоритмов. Помимо двух SAT-режимов включены исходный жадный алгоритм дописывания (**Append**), жадные вставки (**Insertion**), специализированный маршрутизатор **OR-Tools Routing + Guided Local Search** и полная целочисленная модель **OR-Tools CP-SAT**. Два последних метода используют OR-Tools 9.15.6755 через Python; приложение и обе жадные процедуры — Rust.

3.1. Точная скаляризация для OR-Tools

Пусть $n = |J|$, $m = |E|$. Для каждой заявки существует не более одного оплачиваемого входящего перехода. Поэтому корректна общая верхняя оценка

$$D_{\max} = \sum_{j \in J} \max_{e \in E, i \in J \cup \{o_e\}} d_{ij}^e. \quad (36)$$

Недостижимые переходы исключены из реально исполняемых путей; их нулевая фиктивная стоимость не ослабляет верхнюю оценку достижимых маршрутов. Вводятся целые веса

$$W_H = D_{\max} + 1, \quad W_V = (m + 1)W_H, \quad W_U = (n + 1)W_V, \quad (37)$$

и минимизируется

$$C = W_U U + W_V V + W_H H + D. \quad (38)$$

Доказательство эквивалентности. Из $D \leq D_{\max} < W_H$ следует приоритет одной единицы H над любым изменением расстояния. Далее $mW_H + D_{\max} = W_V - 1$, а $nW_V + mW_H + D_{\max} = W_U - 1$. Поэтому единица каждой старшей компоненты перевешивает сумму возможных изменений всех младших. Это не настроенные «штрафные коэффициенты», а точная скаляризация лексикографического порядка. Перед решением проверяется консервативная оценка $n(W_U + W_V) + mW_H + D_{\max} < 2^{62}$. Отчётные метры остаются невзвешенными.

3.2. Routing с Guided Local Search

Для каждого инженера создаётся собственный начальный узел, для окончания — фиктивный сток. Стоимость дуги берётся из его направленной матрицы. Транзит измерения времени на дуге $i \rightarrow j$ равен $p_i + \tau_{ij}^e$; для исходного узла $p_i = 0$, а для перехода в сток транзит равен только p_i . Тем самым учитывается обслуживание последней заявки, но не выдумывается возврат на базу. Время источника фиксировано в s_e , окна относятся к времени прихода/начала сервиса в узле, время стока ограничено f_e . Положительный запас времени позволяет ждать.

Заявки необязательны: одиночная дизъюнкция получает штраф $W_V + W_U$ для срочной заявки и W_V для остальных. Непустой маршрут ранее не использованного инженера оплачивается фиксированной стоимостью W_H . Уже оплаченный персонал добавляет только постоянную $W_H \sum_e h_e$. Разрешённые исполнители фильтруются по навыку и транспорту; у необязательной заявки сохраняется возможность отсутствия исполнителя. При отсутствии совместимых инженеров активность узла фиксируется в ноль.

Стартом служит лучший по F из заново построенных Append и Insertion. Если API не принимает маршрутный старт, используется **PARALLEL_CHEAPEST_INSERTION**; затем включается **GUIDED_LOCAL_SEARCH**. GLS изменяет временную оценку поисковых переходов с помощью штрафов, чтобы выходить из локальных оптимумов; эти штрафы не входят в итоговый C . Возвращается лучший проверенный план относительно исходной цели. Поиск Routing выполняется одним потоком; переход к CP-SAT отключён. Глобальная оптимальность для этого компаратора не заявляется независимо от численного кода статуса Routing. Настройки и смысл ограничений поиска описаны в [официальной документации](#).

3.3. Прямая модель CP-SAT

Используются булевы x_{ej}, u_j, y_e , целые времена $t_j \in [a_j, b_j]$ и отдельная система дуг каждого инженера. В ней один фиктивный узел o одновременно играет роль начала и конца **комбинаторного** цикла; завершающая дуга в o не означает физического возврата.

Ограничение **AddCircuit** охватывает дуги $o \rightarrow j$, $i \rightarrow j$, $j \rightarrow o$, а также петли $j \rightarrow j$ с литералом $\neg x_{ej}$ и петлю $o \rightarrow o$ с литералом $\neg y_e$. Таким образом, невыбранные узлы исключаются петлями, а выбранные образуют единственный цикл через фиктивный узел, то есть открытый маршрут после удаления этого узла. Дополнительно задаются

$$u_j + \sum_e x_{ej} = 1, \quad x_{ej} \leq y_e, \quad \sum_j x_{ej} \geq y_e. \quad (39)$$

Несовместимые назначения равны нулю. При $x_{ej} = 1$ выполняются $t_j \geq s_e$ и $t_j + p_j \leq f_e$. На выбранных дугах действуют

$$o \rightarrow j : t_j \geq s_e + \tau_{o_ej}^e, \quad (40)$$

$$i \rightarrow j : t_j \geq t_i + p_i + \tau_{ij}^e, \quad (41)$$

$$j \rightarrow o : t_j + p_j \leq f_e. \quad (42)$$

Цель — приведённая выше точная сумма C ; метры приписываются только дугам, ведущим к реальным заявкам.

Для уменьшения модели удаляются лишь заведомо невозможные **непосредственные дуги**. Если $\ell_{ei} = \max(a_i, s_e)$, $r_{ej} = \min(b_j, f_e - p_j)$ и $\ell_{ei} + p_i + \tau_{ij}^e > r_{ej}$, дуга $i \rightarrow j$ не нужна. Аналогично проверяется дуга из базы. Отсутствие прямой дуги из базы **не запрещает назначение** x_{ej} : заявка может идти после другого посещения. Это существенно для неметрических матриц.

Стартовые значения времён, назначений и дуг получаются из того же лучшего жадного плана. Установлены `num_search_workers = 1`, `random_seed = 1`, нулевые абсолютный и относительный допуски оптимальности. Лимит передаётся только после полного построения модели с вычетом уже потраченного времени. При истечении бюджета во время построения частичная модель не решается — сохраняется жадный план.

Статус **OPTIMAL** означает глобальную оптимальность этой полной модели; **FEASIBLE** — только наличие допустимого ответа. **UNKNOWN** не означает недопустимость и не является доказательством нижней границы. При сохранённом допустимом старте ответы **INFEASIBLE** или **MODEL_INVALID** считаются ошибкой постановки/реализации. Если ответ поиска хуже стартового плана, возвращается старт; противоречие между худшим ответом **OPTIMAL** и стартом вызывает ошибку. Целочисленное значение выражения цели сверяется с метриками проигрывания, без опоры на округлённый floating-point вывод objective API. Значения статусов определены в [документации CP-SAT](#).

3.4. Контроль сопоставимости

Оба компаратора заново строят собственный жадный старт в каждом запуске, а не получают результат SAT, решения большего бюджета или сохранённый пользовательский план. Построение старта включено в их бюджет. Собственная стартовая эвристика SAT включает дополнительные многостарты и полировку; это часть измеряемой реализации, а не общий внешний oracle. Сравнение поэтому отражает качество готовых вычислительных конвейеров, не «чистую» производительность ядер SAT и CP-SAT при идентичных эвристиках.

Независимый малый перебор проверяет 29 метрических и направленных неметрических примеров: допустимость, лексикографический оптимум CP-SAT, запреты навыков/транспорта, оплату уже использованных инженеров, окончание последнего обслуживания и случаи, когда промежуточная остановка исправляет недостижимость прямого выезда. Дополнительно проверяются остановка при построении, UNKNOWN, ошибочный старт и противоречивое заявление OPTIMAL. В этих проверках Routing должен выдавать допустимый план не хуже своего старта; глобальная гарантия от него не требуется. Основной Rust-набор содержит 21 тест, включая 120 небольших офлайн-случаев и 60 случаев направленных неметрических улиц с независимым перебором.

4. Вычислительный эксперимент

4.1. Входные наборы и происхождение матриц

Использованы все три файла с пометкой «Синтетические данные» из `dataset/`. Контрольные распределения не включались: они не задают тот же проверенный набор исполнителей и не являются сертификатами оптимальности. В частности, независимая проверка контрольного юго-восточного файла ранее обнаружила противоречивый повтор идентификатора; он не используется как эталон.

Набор	Заявки	Срочные	Сервис, мин	Точки графа
Восток	66	8	3560	66
Юго-восток	83	12	4490	76
Югоцентр	56	1	2830	56

Различие числа заявок и точек связано с совпадающими координатами и добавлением общей базы. В каждом наборе после явного исключения E06 и E12 остаются десять инженеров: шесть автомобилей, два пешехода и два велосипеда. Исключённый общественный транспорт не заменяется условной скоростью. Используются оригинальные сгенерированные навыки и смены 540–1439 минут. Сервис: подключение — 70 минут, авария — 80, дозаказ — 20, локальная работа — 30. Обычные окна — двухчасовые с 10:00 до 22:00; в юго-восточном наборе есть также широкое окно 00:01–23:59. Длительность 15-часовой доступности — свойство синтетического импорта, не рекомендация режима труда.

Публичная матричная служба Valhalla отказалась строить юго-восточную матрицу с сообщением `Path distance exceeds the max distance limit: 30000 meters`: набор включает удалённые районы, в том числе Каширу и Ступино. Чтобы не урезать данные, не объявлять такие переходы недостижимыми и не заменять их прямой линией, для всех трёх наборов использован один другой источник: [FOSSGIS/OSRM](#), отдельные серверные профили `routed-car`, `routed-bike`, `routed-foot`.

Для каждого набора выполнено три табличных запроса с `annotations=duration,distance; fallback_speed` не задавался, ответы с такими подстановками отклоняются. Соблюдается интервал не менее 1,1 секунды между началами запросов. Сохранены сырые ответы, URL, время получения, нормализованные матрицы, сценарии и SHA-256. Всего для основных матриц — девять запросов; повторные расчёты используют локальные файлы. Табличный API возвращает метры, поэтому перед нормализацией множитель 1000 из километров не применяется. Время округляется по формуле с запасом одной секунды, одинаково для всех алгоритмов.

Все пары этих снимков достижимы. Направленные значения не усредняются и не заменяются расстояниями по сфере. OSRM привязывает точки к доступным рёбрам: максимальное смещение составляет 303,1 м на Востоке, 83,7 м на Юго-востоке и 187,9 м в Югоцентре. Последующий подход от ребра к объекту не моделируется. Это ещё одна причина не трактовать эксперимент как проверенную адресную логистику. Снимки содержат дорожные стоимости, но не полилинии; геометрия не выдумывается и не нужна целевым функциям. Нормализованные снимки помещены в изолированный `benchmark-кэш`, а не в рабочий `кэш` интерфейса.

4.2. Протокол и измерение времени

Каждая ячейка — один независимый процесс с новым состоянием модели и решателя; ни результат предыдущего бюджета, ни пользовательский шестичеловечный план не используется. Выполнено 15 запусков на набор: две жадные процедуры, `fast` с потолком 1 с, `exact` с 1/5/30/120 с и оба компаратора с такими же четырьмя бюджетами. Всего 45 завершённых планов. Порядок фиксирован, запуски последовательны, конкурирующие тесты, компиляторы и другие решатели во время измерений не запускаются. Аппаратные и файловые кэши специально не очищаются: «холодный старт» означает отсутствие сохранённого поискового состояния, не холодное железо.

Среда: macOS-27.0-arm64-arm-64bit-Mach-O Python 3.13.3 OR-Tools 9.15.6755 Apple M2 (8 физических и 8 логических ядер), 16 ГБ. Для решателей используется один рабочий поток. У CP-SAT явно задан `seed 1`; у CaDiCaL сохранены штатные настройки конфигурации `sat`, а эвристические многостарты детерминированы. Параметры не подбирались отдельно под полученные результаты.

Сеть, построение дорожных матриц и компиляция исключены из бюджета. Внутреннее время включает загрузку стоимостей, построение стартов, модели и поиск; у Python-компараторов отсчёт начинается после импорта библиотек. `wall` измеряется родительским процессом от запуска дочернего до завершения и

дополнительно включает запуск интерпретатора/бинарника, сериализацию и завершение. Дополнительное общее контрольное проигрывание после запуска вынесено за оба измерения; внутренние проверки решателей остаются внутри их времени. Бюджеты кооперативные, не жёсткое ограничение ОС.

Сторонние конвейеры оплачивают внутри своего бюджета два дополнительных запуска Rust для жадных стартов и повторную загрузку снимка; Routing также вызывает Python-функции стоимости. SAT строит свои данные непосредственно в одном процессе. Это особенно важно при лимите 1 с: сравниваются данные реализации целиком, а не одинаковая стоимость модельного фронтенда. В таблицах приведены и лимит, и обе фактические длительности.

4.3. Полные результаты

В таблицах U — число срочных пропусков, V — всех пропусков, H — дневной штат, D — метры, переведённые в километры. Сравнивать строки необходимо по (U, V, H, D) , а не по одному D . Знак * после H означает, что этот конвейер доказал глобальный минимум штата при оптимальных старших целях; в fast большого размера доказательство только по пулу таким знаком не отмечается. Знак † означает возврат общего жадного старта: лучшего лексикографического плана поиск не выдал. Это не обязательно UNKNOWN — найденный допустимый ответ тоже может не улучшить старт.

4.3.1. Восток

Метод	Лимит, с	U / V	H	D, км	внутр. / wall, с
Append	—	4 / 27	10	215,644	0,002 / 0,006
Insertion	—	0 / 0	9	265,409	0,002 / 0,006
SAT fast	1	0 / 0	8	206,524	0,026 / 0,030
SAT exact	1	0 / 0	7	186,712	0,900 / 0,907
SAT exact	5	0 / 0	6*	238,403	4,909 / 4,956
SAT exact	30	0 / 0	6*	240,686	29,913 / 29,938
SAT exact	120	0 / 0	6*	238,403	119,903 / 119,930
Routing GLS	1	0 / 0	7	161,261	1,001 / 1,662
Routing GLS	5	0 / 0	7	139,586	5,001 / 5,413
Routing GLS	30	0 / 0	7	121,892	30,001 / 30,426
Routing GLS	120	0 / 0	7	121,892	120,000 / 120,482
CP-SAT†	1	0 / 0	9	265,409	1,003 / 1,481
CP-SAT†	5	0 / 0	9	265,409	5,010 / 5,406
CP-SAT	30	0 / 0	9	258,228	30,023 / 30,418
CP-SAT	120	0 / 0	9	203,721	120,028 / 120,441

4.3.2. Юго-восток

Метод	Лимит, с	U / V	H	D, км	внутр. / wall, с
Append	—	0 / 31	10	985,582	0,003 / 0,007
Insertion	—	0 / 6	10	1139,084	0,003 / 0,007
SAT fast	1	0 / 4	9	1049,707	0,030 / 0,035
SAT exact	1	0 / 0	9	1000,966	0,936 / 0,950
SAT exact	5	0 / 0	8	888,377	4,908 / 4,925
SAT exact	30	0 / 0	8	905,636	30,023 / 30,037
SAT exact	120	0 / 0	8	888,377	119,908 / 119,925
Routing GLS	1	0 / 0	9	533,611	1,000 / 1,650
Routing GLS	5	0 / 0	9	531,974	5,001 / 5,393
Routing GLS	30	0 / 0	9	521,603	30,002 / 30,395
Routing GLS	120	0 / 0	9	521,603	120,001 / 120,400
CP-SAT†	1	0 / 6	10	1139,084	1,034 / 1,512
CP-SAT†	5	0 / 6	10	1139,084	5,031 / 5,428
CP-SAT	30	0 / 6	10	936,487	30,080 / 30,482
CP-SAT	120	0 / 6	10	936,266	120,033 / 120,438

4.3.3. Югоцентр

Метод	Лимит, с	U / V	H	D, км	внутр. / wall, с
Append	—	0 / 14	10	265,371	0,001 / 0,005
Insertion	—	0 / 0	7	233,368	0,001 / 0,005
SAT fast	1	0 / 0	6	204,462	0,029 / 0,033
SAT exact	1	0 / 0	5	253,621	0,900 / 0,907
SAT exact	5	0 / 0	5	253,621	4,900 / 4,908
SAT exact	30	0 / 0	5	253,621	29,900 / 29,910
SAT exact	120	0 / 0	5	253,621	119,917 / 119,925
Routing GLS	1	0 / 0	6	151,735	1,001 / 1,465
Routing GLS	5	0 / 0	5	160,253	5,001 / 5,394
Routing GLS	30	0 / 0	5	159,368	30,001 / 30,398
Routing GLS	120	0 / 0	5	159,221	120,002 / 120,410
CP-SAT†	1	0 / 0	7	233,368	0,798 / 1,302
CP-SAT	5	0 / 0	7	232,146	5,019 / 5,418
CP-SAT	30	0 / 0	7	189,940	30,018 / 30,415
CP-SAT	120	0 / 0	6	210,830	120,018 / 120,424

Полные статусы, области доказательств, границы этапов, списки посещений и исходные ответы доступны в [reports/road-benchmark/runs/](#). Все 45 планов дополнительно прошли Rust-проигрывание, и их метрики совпали с заявленными.

4.4. Сравнение и интерпретация

Восток. При 1 с Routing GLS лучше exact по метрам при одинаковом полном покрытии и семи инженерах: 161,261 против 186,712 км. При 5 с exact находит шесть инженеров и 238,403 км, причём глобальный минимум штата доказан. GLS даже при 120 с остаётся на семи инженерах и 121,892 км. По заданной цели exact лучше: уменьшение штата важнее дополнительных 116,511 км. Это наглядно показывает, почему одна колонка расстояния не определяет победителя. CP-SAT за 120 с сохраняет девять инженеров; его улучшение расстояния до 203,721 км не компенсирует проигрыш по штату.

Юго-восток. Insertion пропускает шесть заявок, fast — четыре. Eхact и GLS обслуживают все 83. При 1 с они используют девять инженеров, но GLS существенно лучше по метрам: 533,611 против 1000,966 км. Начиная с 5 с ехact использует восемь инженеров (888,377 км), тогда как GLS остаётся на девяти (521,603 км к 30 с). Ехact выигрывает по третьей компоненте, но **минимальность восьми не доказана**: глобальный интервал штата при полном покрытии остаётся $7 \leq H^* \leq 8$. CP-SAT не устраняет шесть пропусков даже за 120 с, поэтому его расстояния нельзя сравнивать с полным обслуживанием как равноценное качество.

Югоцентр. При 1 с ехact выигрывает по штату: пять против шести у GLS. Уже при 5 с GLS также находит пять, но с 160,253 км против 253,621 км у ехact; при 120 с GLS улучшает результат до 159,221 км, примерно на 37,2 % короче ехact при равных старших целях. Все четыре прогона ехact возвращают одинаковый вектор. Граница штата остаётся $4 \leq H^* \leq 5$: поиск занят нерешённым этапом персонала, поэтому точный SAT-этап метров не начинается. Полировка допустимого плана работает, но не заменяет специализированного маршрутного поиска. Здесь длительный ехact **не является лучшим практическим ответом** среди сравниваемых конвейеров.

При одинаковых лимитах ехact лексикографически лучше GLS в 7 из 12 пар, GLS — в 5. Это описательный счёт на трёх экземплярах, не статистическая оценка вероятности победы. Для CP-SAT принятая прямая однопоточная модель слабее обоих основных конвейеров на этих данных; результаты не означают, что иной CP-SAT-моделью нельзя получить существенно больше.

4.5. Скорость, размер модели и доказательства

Fast заканчивает за 26–30 мс внутреннего времени (30–35 мс wall), задолго до потолка 1 с: останавливает прежде всего предел усилий SAT. Цена скорости — восемь вместо шести инженеров на Востоке, четыре пропуска на Юго-востоке и шесть вместо пяти инженеров в Югоцентре. Он улучшает простые жадные планы, но не обеспечивает ни полное покрытие на всех трёх случаях, ни глобальное доказательство. Это пригодный быстрый допустимый ответ, а не эквивалент полной модели.

В следующей таблице показаны счётчики созданной приложением CNF: fast при 1 с и ехact при 120 с. Клаузы, выученные внутренне CaDiCaL, в этот счётчик не входят. Значения относятся к достигнутому этапу, а не к единственной фиксированной заранее модели.

Набор	Режим	Маршрутов в пуле	Переменных	Клауз
Восток	fast	819	4540	18372
Восток	exact 120	—	368881	1457764
Юго-восток	fast	909	5766	23146
Юго-восток	exact 120	—	99694	824426
Югоцентр	fast	695	3967	16134
Югоцентр	exact 120	—	37164	255609

На Востоке ехact доходит до полного слоя переходов и арифметики метров; в двух других наборах он остаётся на этапе штата. Поэтому размер CNF не обязан возрастать вместе с числом заявок: 66 заявок здесь дают более крупную достигнутую модель, чем 83.

Ни один из 45 запусков не доказывает все четыре компоненты цели. Три глобальных доказательства минимального штата получены для Востока — ехact 5/30/120 с. Везде, где все заявки обслужены, нулевые пропуски автоматически минимальны; это не доказывает минимум штата или метров. В fast выданные этапные границы относятся только к пулу. В CP-SAT все три односекундных запуска имеют UNKNOWN и возвращают общий жадный старт; при 5 с Восток и Юго-восток также сохраняют старт, хотя уже имеют статус FEASIBLE. Это честный возврат допустимого решения, не «решение задачи CP-SAT до оптимума».

4.6. Почему больше времени не всегда даёт лучшую строку

Прогоны с разными лимитами не являются продолжениями одного поиска. На Востоке ехact 30 с возвращает 240,686 км против 238,403 км при 5 с, а на Юго-востоке — 905,636 против 888,377 км при том же штате. Внутри одного запуска лучший проверенный план не ухудшается; между независимыми процессами влияют ограниченные по времени начальные эвристики, момент остановки и полировка. Наблюдаемая немонотонность не оправдывает выбор только удачного измерения: в отчёте сохранены все строки, в том числе худшие при большем бюджете.

Практический вывод — разделять три результата: быстрое получение допустимого плана, улучшение маршрутов и доказательство невозможности меньшего штата. Eхast полезен для второго/третьего в зависимости от экземпляра, но слово «exаst» означает полноту пространства поиска и возможность доказательства, а не гарантированный оптимальный ответ за заданные секунды. При дальнейшей доработке имеет смысл усиливать полировку фиксированного штата и стартовые решения, не превращая их в ограничения полной SAT-модели. Такой вариант в текущие измерения не включён.

5. Ограничения и воспроизводимость

Это три небольших экземпляра одной прикладной семьи, по одному запуску каждой конфигурации. Не оценены дисперсия, доверительные интервалы, масштабирование и чувствительность к seed; нет статистического основания объявлять один решатель универсально лучшим. Фиксированный порядок запусков, фон ОС и зависящее от дедлайна поведение вносят вариативность. Использована одна прямая модель CP-SAT и стандартный GLS, без систематического подбора параметров или сравнения всех специализированных VRPTW-решателей. Слабый результат этой модели CP-SAT не является общим выводом о возможностях OR-Tools.

Глобальные гарантии относятся к сохранённым целочисленным данным и лексикографическому порядку. Они не покрывают реальные неизвестные координаты, дорожный трафик, смену дорожного графа, перерывы, вместимость, складские запасы, дополнительные трудовые нормы или ошибки нормативных длительностей. Инкрементальное перепланирование реализовано и проверено отдельно, но эти 45 запусков — планы с начала дня без событий.

Полный материал эксперимента:

- `reports/dispatch-report.typ`, `reports/article/` — исходники и данные статьи;
- `reports/road-benchmark/manifest.json` — происхождение, хэши сценариев/матриц;
- `provider/`, `snapshots/`, `inputs/` — сырые ответы, фиксированные стоимости и сценарии;
- `runs/`, `environment.json`, `results.csv` — измерения, статусы и среда;
- `scripts/prepare-road-benchmark.py`, `road-benchmark.py` — подготовка и запуск;
- `scripts/bench_comparators.py`, `test_bench_comparators.py` — внешние модели и проверка перебором.

Обычный повтор с сохранёнными снимками не обращается к сети. `prepare-road-benchmark.py` нужен только при создании/восстановлении входов; он повторно использует сырые ответы, если они уже есть. Сборка статьи из уже зафиксированных результатов не запускает решатели:

```
python3 scripts/summarize-road-benchmark.py
typst compile reports/dispatch-report.typ reports/dispatch-report.pdf
```

Для нового вычислительного эксперимента требуются Rust и отдельная Python-среда:

```
cargo build --release --locked
uv venv --python 3.13.3 .bench-venv
uv pip install --python .bench-venv/bin/python \
  -r scripts/benchmark-requirements.txt
.bench-venv/bin/python scripts/test_bench_comparators.py
.bench-venv/bin/python scripts/test_road_benchmark.py
.bench-venv/bin/python scripts/road-benchmark.py
python3 scripts/summarize-road-benchmark.py
```

Повтор перезаписывает текущие измерения: для сохранения исторического результата предварительно скопируйте каталог отчёта. Новый эксперимент сохраняет снимок реально исполняемых скриптов и ядра; `--resume` проверяет совпадение хэшей кода и записанных параметров среды до перезаписи метаданных. Хэш имени узла не удостоверяет физическую идентичность компьютера. Исторические прогоны выполнены архивной версией `gimp`; защита возобновления усилена после них без изменения алгоритмов и измерений. Исходный архив и его хэши сохранены. Числа и аналитический текст настоящей статьи описывают исходный эксперимент: после нового измерения перед пересборкой PDF необходимо сверить и обновить выводы, а не автоматически переносить их на новые строки.

6. Источники и программные материалы

1. Исходный код `dispatch-sat`: `src/model.rs`, `src/sat.rs`, `src/fast.rs`, `src/routing.rs`, `src/import.rs`, `src/norms.rs`, `src/app.rs`. Формулы настоящей статьи соответствуют этой реализации; сравниваемые сценарии и хэши сохранены в артефактах эксперимента.

2. [CaDiCaL SAT Solver](#). Репозиторий и инкрементальный интерфейс с предположениями. Rust-связка `cadical` 0.1.16.
3. [Google OR-Tools: Routing Options](#). Лимиты, локальный поиск и Guided Local Search.
4. [Google OR-Tools: CP-SAT Solver](#). Целочисленные модели и смысл статусов.
5. [OSRM HTTP API: Table service](#). Длительности, длины соответствующих путей, отсутствующие маршруты и явный `fallback_speed`.
6. [FOSSGIS routing server](#): происхождение данных, профили и правила использования. Данные © OpenStreetMap contributors, ODbL. Исправить карту. Публичный сервис не имеет SLA; эксплуатационные партии требуют собственной инфраструктуры.
7. [Valhalla: Matrix API](#). Источник уличных снимков рабочего приложения; в данном сравнении заменён единообразно на OSRM из-за ограничения публичной матрицы.